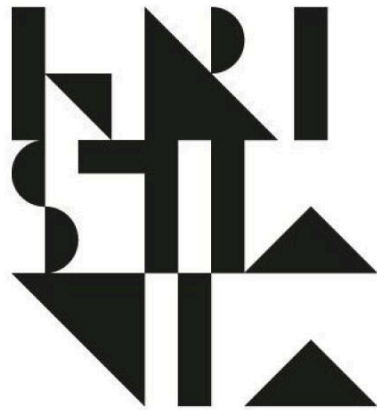




Høyskolen
Kristiania

Candidate numbers: 83, 162
Group number: 130

Date: 24.03.2026 - 21.04.2026

Course: PG4200 - Algorithms and Data Structures

Table of Contents

Supporting classes.....	4
Main.....	4
ReadAlcoholValues.....	4
SortBenchmark.....	4
PrintResults.....	5
Problem 1: Bubble Sort.....	6
Introduction:.....	6
Implementation:.....	6
Problem 1: Optimized Bubble Sort.....	9
Introduction:.....	9
Implementation:.....	9
Time Complexity.....	11
Conclusion:.....	12
Problem 2: Insertion Sort.....	13
Implementation.....	13
Time complexity.....	15
The effect of shuffling the dataset.....	16
Problem 3: Merge Sort.....	17
Implementation.....	17
Shuffled vs non-shuffled.....	20
Problem 4: QuickSort.....	21
Introduction.....	21
Implementation.....	21
Problem 4b: QuickSort Pivot Strategies.....	26
Does the number of comparisons change depending on the pivot strategy?.....	27
Which pivot selection strategy works best for this dataset, and why?... 27	
References.....	29

Supporting classes

Main

Main reads the alcohol values from the two CSV files using `ReadAlcoholValues.readFromFile`. It then removes duplicate values from the combined list to get a clean array of alcohol percentages. It sets the number of benchmark runs to 1000 and passes the array into `SortBenchmark.runBenchmark` along with the chosen sorting algorithm. To choose the sorting algorithm you want to run, uncomment one or multiple lines.

ReadAlcoholValues

`ReadAlcoholValues` handles reading and preparing the dataset. The `readFromFile` method accepts one or more file paths using varargs, making it possible to pass in multiple CSV files in a single call. For each file, it opens a `Scanner`, skips the header row, and then reads each line, splitting it by semicolons. It extracts the value at index 10, which is the alcohol content column, and then parses it to a double, adding it to an `ArrayList`. The `try-with-resources` block ensures each file is closed after reading. The `removeDuplicates` method then takes that list and uses a stream to filter out duplicate values with `distinct()`, converts them from `Double` objects to primitive double values, and returns the result as an array. This gives us a clean array of unique alcohol values ready to be passed into the benchmark.

SortBenchmark

The `SortBenchmark` class is responsible for measuring the performance of a sorting algorithm by running it multiple times on both the original array and a shuffled copy. It takes in a sorting algorithm, an array, and the number of runs, which is 1000. Before the actual benchmarking begins, it performs 200 warm-up runs without measuring. This is done because the Java Virtual Machine optimizes code as it runs. The first few executions are slower while the JVM compiles the code (Mytnik, 2024). Running it 200 times first ensures the actual measurements are not affected by this.

For each of the specified runs, it clones the original array twice. One stays as it is and the other gets shuffled using the Fisher-Yates shuffle method from the `FisherYatesShuffle` class. This cloning is necessary because the sorting algorithm sorts the array in place, so without a fresh copy, each run would receive an already-sorted array. It sorts both, recording the time in nanoseconds using `System.nanoTime()`, the number of swaps/shifts/merge operations,

and the number of comparisons for each. After all runs are complete, it calculates the averages by dividing the totals by the number of runs and passes everything to `PrintResults.print` along with a sorted copy of the array.

PrintResults

The `PrintResults` class is a utility class that formats and displays the benchmark results to the console. Its `print` method takes in the algorithm name, the sorted array, and the metrics for both the original and shuffled arrays. It prints everything in a structured format separated by divider lines. This includes the average time it takes to sort in nanoseconds, then the number of swaps/shifts/merge operations, then the number of comparisons, and finally the sorted array.

Problem 1: Bubble Sort

Introduction:

BubbleSort is a basic sorting algorithm, and gets its name as the biggest values “bubbles” to the top of the list, (W3Schools, n.d.-a). Bubble sort is an iterative algorithm that sorts the elements in a dataset by comparing adjacent elements with another. While this algorithm is very intuitive and easy to learn, its $O(n^2)$ time complexity makes it very inefficient for large datasets.

Below is a simple, step-by-step breakdown of the algorithm.

Implementation:

Non-optimized Bubble Sort

```
5      @Override 4 usages
6      public void algorithm(double[] arr) {
7          int n = arr.length;
8          nrOfSwaps = 0;
9          nrOfComparisons = 0;
10
11         for (int i = 0; i < n - 1; i++) {
12             for (int j = 0; j < n - 1 - i; j++) {
13                 nrOfComparisons++;
14                 if (arr[j] > arr[j + 1]) {
15                     double temp = arr[j];
16                     arr[j] = arr[j + 1];
17                     arr[j + 1] = temp;
18                     nrOfSwaps++;
19                 }
20             }
21         }
22     }
```

Line 6: Declaration of **bubbleSort()** method, the algorithm only takes in the unsorted array.

Line 7: Simple auxiliary space initialized to store the array's length, preventing the need to repeatedly call the array's ".length" method.

Line 8-9: Lines 8 and 9 reset the swap and comparison counter to "0". This is done at the very beginning to ensure accurate counting of the two.

Line 11: The outer for-loop will run $n - 1$ times, where n equals the length of the array. This means that with every iteration of this loop, an element in the list has been sorted to its final spot.

Line 12: The inner for-loop, this loop does all the heavy lifting in this algorithm. For every iteration of the outer loop, this inner loop will compare two and two elements with each other. Another note on this line is that the loop runs $n - 1 - i$ times, meaning for every iteration of the outer loop, the number of iterations for the inner loop decreases. A truly "non-optimized" version of this algorithm would run $n - 1$ times. Which would iterate through every element of the array even if the elements are sorted.

Line 13: Incrementing the nrOfComparisons, this is done within the inner for-loop to get an accurate number.

Line 14: Checking if the value at the current index is larger than the value of the next index, i.e. comparing adjacent elements and swapping them if the check returns true.

Line 15: Initializing a temporary double variable temp to store the current index's value. This is because we don't want to lose the value while swapping the values.

Line 16: Storing the value at the next index into the current index.

Line 17: Replacing the value of the next index with the value stored in temp. The swapping is now finished, both elements swapped places with each other.

Line 18: Incrementing the nrOfSwaps, this is being done last in the if-block to make sure we get an accurate number.

Here is a simplified equation with the result showing the time complexity for bubble sort. I have intentionally left out all the steps that are not necessary for bubble sort to run, these are

code lines: 8-9, 13 and 18. The reason for doing this is that i do not consider it necessary to include them here, the worst and average time complexity will nevertheless be $O(n^2)$.

$$\begin{aligned} & 1 + n * n \\ &= 1 + n^2 = O(n^2 + 1) = O(n^2) \end{aligned}$$

Problem 1: Optimized Bubble Sort

Introduction:

The Optimized Bubble Sort algorithm is very similar to the non-optimized version. The core function of the method is exactly the same, the main difference is that it utilizes a boolean flag variable. The function of the “isSwapped” flag is to break out of the algorithm if there has not been any swaps in that iteration, meaning, the elements are already sorted. The inner for-loop runs “- i” because the largest elements have already bubbled to the top of the array.

Implementation:

```
6 ①↑@ public void algorithm(double[] arr){
7      int n = arr.length;
8      nrOfSwaps = 0;
9      nrOfComparisons = 0;
10     boolean isSwapped;
11     for (int i = 0; i < n - 1; i++){
12         isSwapped = false; //Set to false by default
13         for (int j = 0; j < n - 1 - i; j++){
14             nrOfComparisons++;
15             if (arr[j] > arr[j + 1]){
16                 double temp = arr[j];
17                 arr[j] = arr[j + 1];
18                 arr[j + 1] = temp;
19                 nrOfSwaps ++;
20                 isSwapped = true;
21             }
22         }
23         if (!isSwapped){
24             break; //True = break out of loop
25         }
26     }
27 }
```

Line 6: Declaration of **bubbleSort()** method, the algorithm only takes in the unsorted array.

Line 7: Simple auxiliary space initialized to store the array's length, preventing the need to repeatedly call the array's ".length" method.

Line 8-9: Lines 7 and 8 reset the swap and comparison counter to "0". This is done at the very beginning to ensure accurate counting of the two.

Line 10: Declaration of the boolean flag boolean isSwapped.

Line 11: The outer for-loop will run $n - 1$ times, where n equals the length of the array. This means that with every iteration of this loop, an element in the list has been sorted to its final spot.

Line 12: Setting the isSwapped to false at the top of the outer for-loop, this resets the flag and enters the inner for-loop.

Line 13: The inner for-loop will run $n - 1 - i$ times, this means that for each iteration of the outer loop, the number of iterations the inner loop decreases by one.

Line 14: Incrementing the nrOfComparisons, this is done within the inner for-loop to get an accurate number.

Line 15: Checking if the value at the current index is larger than the value of the next index, i.e. comparing adjacent elements and swapping them if the check returns true.

Line 16: Initializing a temporary double variable, temp to store the current index's value. This is because we don't want to lose the value while swapping the values.

Line 17: Storing the value at the next index into the current index.

Line 18: Replacing the value of the next index with the value stored in temp. The swapping is now finished, both elements swapped places with each other.

Line 19: Incrementing the nrOfSwaps, this is being done last in the if-block to make sure we get an accurate number.

Line 20: Since a swap has been done, the flag is set to true.

Line 23-24: Checking if the isSwapped is still false, and if it is, break out of the algorithm.

This is the core of the optimized version of bubble sort, because it allows the algorithm to break out of the nested for-loops, minimizing the number of unnecessary swaps and comparisons being done.

Here is a simplified equation with the result showing the time complexity for bubble sort. we have intentionally left out all the steps that are not necessary for the optimized bubble sort to run, these are code lines: 7-8, 14, and 19. The reason for doing this is that we don't think it necessary to include them here, the worst and average time complexity will nevertheless be $O(n^2)$.

$$\begin{aligned} & 1 + 1 + n * n \\ & = 2 + n^2 = O(n^2 + 2) = O(n^2) \end{aligned}$$

Time Complexity

Bubble sort and the optimized bubble sort algorithms have slightly different time complexities. Bubble sort has a best, average and worst time complexity of $O(n^2)$, this is because of the nested for-loops which evolve quadratically. In the case of the optimized version however if the array is already sorted, the isSwapped flag is never changed, which means no swaps were made that pass. This will result in a best time complexity of $O(n)$.

Shuffling the list has a noticeable impact on the performance of both algorithm's, although their average/worst time complexity remains $O(n^2)$. Theoretically there is a chance, however astronomically small, that shuffling the array results in all values being placed in their correct, sorted place. In which case the optimized version's best time complexity will be $O(n)$. But for the sake of this exam we will conclude that the time complexity does not change if the array is shuffled beforehand.

Since we first warm-up the JVM with 200 runs, then calculate an average time (in nanoseconds) of 1000 runs, we can get a very accurate number of elapsed time. Even though this is not the focus of this exam, it is a very interesting point. As we can see below, when the array is shuffled it takes roughly 2,6 times longer for the algorithm to complete the

sorting. This can only be because when the elements in the array are shuffled, the algorithm needs to swap more elements than the original array. The number of swaps went up nearly 500 times on the shuffled runs. The number of comparisons stays the same no matter what because the number of elements in the algorithm doesn't change.

```
--- BubbleSort ---
Average Time (Original): 6860 ns
Average Time (Shuffled): 18097 ns
=====
Number of Swaps/Shifts/Merge-operations (Original): 2607
Number of Swaps/Shifts/Merge-operations (Shuffled): 3057
=====
Number of Comparisons (Original): 6105
Number of Comparisons (Shuffled): 6105
Sorted: [8.0, 8.4, 8.5, 8.6, 8.7, 8.8, 8.9, 9.0, 9.05, 9.1,
=====
```

When it comes to the optimized version of bubble sort, we can see that the average time it took to run the algorithm 1200 times nearly doesn't differ from the non-optimized version. That's not really surprising, considering that since shuffling the array doesn't change the time complexity for this algorithm. it still runs $O(n^2)$ times.

```
--- Optimized BubbleSort ---
Average Time (Original): 6796 ns
Average Time (Shuffled): 18972 ns
=====
Number of Swaps/Shifts/Merge-operations (Original): 2607
Number of Swaps/Shifts/Merge-operations (Shuffled): 3043
=====
Number of Comparisons (Original): 5895
Number of Comparisons (Shuffled): 6019
Sorted: [8.0, 8.4, 8.5, 8.6, 8.7, 8.8, 8.9, 9.0, 9.05, 9.1,
=====
```

Conclusion:

Both bubble sort algorithms sort the array correctly, though out of the four evaluated in this exam is the slowest. Both having an average and worst case time complexity of $O(n^2)$, and the optimized version having a best case time complexity of $O(n)$.

Problem 2: Insertion Sort

Insertion sort works by splitting an array into two sub-arrays. The left sub-array holds the sorted values, and the right holds the unsorted values. It takes the first value from the unsorted part of the array and moves it to its correct position in the sorted part. This gets repeated until the entire array is sorted (W3Schools, n.d.-b). Insertion sort is efficient for small and nearly sorted lists, and it is easy to implement. The biggest weakness is its efficiency for large lists. Most sorting algorithms are more efficient in most cases (GeeksForGeeks, 2026a).

Implementation

```
5      @Override 4 usages
6  @ public void algorithm(double[] arr) {
7      int n = arr.length;
8      nrOfShifts = 0;
9      nrOfComparisons = 0;
10
11     for (int i = 1; i < n; i++) {
12         double key = arr[i];
13         int j = i - 1;
14
15         while(j >= 0){
16             nrOfComparisons++;
17             if (arr[j] > key){
18                 arr[j + 1] = arr[j];
19                 nrOfShifts++;
20                 j--;
21             } else {
22                 break;
23             }
24         }
25         arr[j+1] = key;
26     }
27 }
```

Line 6: Method declaration, the method takes in the unsorted array as a parameter.

Line 7: Stores the length of the array in a variable `n`, preventing the need to repeatedly call `arr.length`.

Line 8-9: Resets both counters to zero at the start of each sort, so they only reflect the most recent run.

Line 11: The outer for-loop starts at index 1 and runs to `n-1`. It starts at 1 because the first element is considered already sorted.

Line 12: Stores the current element in a variable named `key`. This is the value that will be inserted into the correct position.

Line 13: Sets `j` to the index just before `i`, which is the last element of the already-sorted portion of the array. This is the starting point for comparing backwards through the sorted section.

Line 15: The while-loop runs as long as `j` has not gone past the start of the array. In the worst case, for each element at position `i`, this loop runs `i` times.

Line 16: Increments the comparison counter every time we check an element against `key`.

Line 17: Checks if the current element is greater than `key`. If it is, it needs to be shifted.

Line 18: Shifts the element one position to the right to make room for `key`.

Line 19: Increments the shift counter since a shift was performed.

Line 20: Decrements `j` to compare the next element to the left.

Line 21-22: If the current element is not greater than `key`, the correct position has been found, so `break` exits the while-loop.

Line 25: Places the `key` into its correct position at `j+1`.

Time complexity

To calculate the time complexity of our insertion sort, we can count how many times each part of the code runs. Lines 7-9 each run once, giving us +1. Lines 12, 13 and 25 are inside the outer for-loop which runs from 1 to $n-1$, so each contributes + n . The while-loop on line 15 is nested inside the for-loop, and how many times it runs depends on the order of the data. The while-loop compares the current element against the already-sorted part of the array and shifts elements to the right to make room for insertion. The more elements that need to be shifted, the longer the algorithm takes.

If the array is already sorted in ascending order, the while-loop enters once per element, but every `arr[j] > key` check is false, so it breaks without shifting anything. The algorithm just walks through the array once doing comparisons, giving us a best case time complexity of $O(n)$ (GeeksForGeeks, 2025c).

If the array is in reverse order, we get $1 + 1 + 1 + n + n + n + n^2 + n = n^2 + 4n + 3$. Since the n^2 dominates, the $4n$ and 3 are dropped. This results in a worst case time complexity of $O(n^2)$ (W3Schools, n.d.-c). The average case time complexity is the same as the worst case. Since our dataset is neither sorted nor reverse sorted, the time complexity is $O(n^2)$. Even though the number of shifts is not the same as if it was reversed, the time complexity is still quadratic (GeeksForGeeks, 2025c).

```
--- Insertion Sort ---
Average Time (Original): 3800 ns
Average Time (Shuffled): 4517 ns
=====
Number of Swaps/Shifts/Merge-operations (Original): 2607
Number of Swaps/Shifts/Merge-operations (Shuffled): 3054
=====
Number of Comparisons (Original): 2711
Number of Comparisons (Shuffled): 3160
Sorted: [8.0, 8.4, 8.5, 8.6, 8.7, 8.8, 8.9, 9.0, 9.05, 9.1,
=====
```

The effect of shuffling the dataset

To test how the input order affects performance, we also ran the sort on a shuffled version of the dataset. The time complexity does not change, it is still $O(n^2)$. However, we found that the shuffled dataset was slightly slower to sort than the original. As we can see, the number of shifts and comparisons were higher in the shuffled dataset. This is because our original dataset was not perfectly sorted but still had some degree of order to it. Shuffling removes that existing order, pushing the number of while-loop iterations closer to the average case and resulting in a higher number of shifts, and a slightly longer runtime.

Problem 3: Merge Sort

Merge sort uses the divide and conquer method by recursively splitting an array into smaller arrays until each array contains only one element. It then recursively merges the sub-arrays by putting the lowest value first. This is repeated until there is only one sorted array left (W3Schools, n. d.-d) Because of its $O(n \log n)$ worst-case time complexity, it is well suited for large datasets. However, it requires additional memory to store the sub-arrays during sorting, and it is generally slower than quick sort (GeeksForGeeks, 2025a).

Implementation

```
5      @Override 4 usages
6      public void algorithm(double[] arr){
7          nrOfMergeOperations = 0;
8          nrOfComparisons = 0;
9          mergeSortRecursive(arr);
10     }
11
12     @
13     private void mergeSortRecursive(double[] arr) { 3 usages
14         int n = arr.length;
15
16         if (n > 1) {
17             int midIndex = n / 2;
18             double[] left = new double[midIndex];
19             double[] right = new double[n - midIndex];
20
21             System.arraycopy(arr, srcPos: 0, left, destPos: 0, midIndex);
22             System.arraycopy(arr, midIndex, right, destPos: 0, length: n - midIndex);
23
24             mergeSortRecursive(left);
25             mergeSortRecursive(right);
26             merge(arr, left, right);
27             nrOfMergeOperations++;
28         }
29     }
```

```

30 @ private void merge(double[] arr, double[] left, double[] right){ 1 usage
31
32     int leftSize = left.length;
33     int rightSize = right.length;
34     int i = 0, j = 0, k = 0;
35
36     while (i < leftSize && j < rightSize){
37         nrOfComparisons++;
38         if(left[i] <= right[j]){
39             arr[k] = left[i];
40             i++;
41         }
42         else {
43             arr[k] = right[j];
44             j++;
45         }
46         k++;
47     }
48
49     while (i < leftSize){
50         arr[k] = left[i];
51         i++;
52         k++;
53     }
54     while (j < rightSize){
55         arr[k] = right[j];
56         j++;
57         k++;
58     }
59 }

```

Line 6: Method declaration, the method takes in the unsorted array as a parameter.

Line 7-8: Resets both counters to zero so they only reflect the most recent sort.

Line 9: Calls **mergeSortRecursive(arr)** to begin the divide and conquer process.

Line 12: Declaration of the mergeSortRecursive method, which handles the divide step.

Line 13: Stores the length of the array in a variable n.

Line 15: Checks if the array has more than one element. If it only has one, it is already sorted and the method returns without doing anything.

Line 16: Calculates the midpoint to determine where to split the array.

Line 17-18: Creates two new arrays, left and right, to hold each half.

Line 20-21: Copies elements from the original array into left and right using System.arraycopy.

Line 23-24: Recursively calls itself on each half, continuing to split until every sub-array has one element.

Line 25: Calls **merge(arr, left, right)** to combine the two sorted halves back together.

Line 26: Increments the merge operation counter, allowing us to track the total number of merge operations.

Line 30: Declaration of the merge method, which combines two sorted halves back into the original array.

Line 32-33: Stores the length of the left and right arrays.

Line 34: Initializes three index variables. i for the left array, j for the right array, and k for the original array.

Line 36-47: The while-loop walks through both halves at the same time, incrementing the comparison counter each time. It compares the current element from each side and copies whichever is smaller into arr, then moves that pointer and k forward. This continues until one of the halves runs out of elements.

Line 49-53: Copies any remaining elements from the left array into arr. This runs if the right array runs out of elements first.

Line 54-58: Copies any remaining elements from the right array into arr. This runs if the left array runs out of elements first.

```

--- MergeSort ---
Average Time (Original): 4632 ns
Average Time (Shuffled): 6029 ns
=====
Number of Swaps/Shifts/Merge-operations (Original): 110
Number of Swaps/Shifts/Merge-operations (Shuffled): 110
=====
Number of Comparisons (Original): 600
Number of Comparisons (Shuffled): 617
Sorted: [8.0, 8.4, 8.5, 8.6, 8.7, 8.8, 8.9, 9.0, 9.05, 9.1,
=====

```

Shuffled vs non-shuffled

The number of merge operations depends only on the size of the array, not the order of the elements. Every time `mergeSortRecursive` is called on an array larger than one element, it splits it in half and calls `merge` once. This happens regardless of what values are in the array, and the structure of the recursion is identical no matter how the data is ordered. With a dataset of 111 elements, the number of merge operations is $n-1 = 110$ (W3Schools, n.d.-e). When running the benchmark 1000 times, both the shuffled and non-shuffled datasets resulted in exactly 110 merge operations, confirming that the number does not change.

The results showed 600 comparisons for the original array and 617 for the shuffled array. Both fall within the expected $O(n \log n)$ range for merge sort, which always performs the same number of merges but a variable number of comparisons depending on input order (W3Schools, n.d.-e). This difference comes from the while-loop in lines 36-47, which compares elements from both halves one by one and increments the comparison counter each iteration. The loop runs until one half is out of elements, at which point the remaining elements from the other half are copied over without any comparisons. In the original array, the arrangement of elements causes one half to run out sooner during many of the merges, meaning the loop exits earlier and the remaining elements get copied. In the shuffled array, the elements from the two halves are more interleaved, so the while-loop has to keep comparing for longer before one side runs out of elements, resulting in 17 additional comparisons.

Problem 4: QuickSort

Introduction

QuickSort is widely considered one of the most efficient sorting algorithms. It is a “Divide and Conquer” algorithm and utilizes a pivot point to determine where to divide the dataset. The Divide and Conquer principle is as follows: Divide the dataset into two sub-arrays based on the chosen pivot point; now, each element is compared to the pivot. If the element is smaller than the pivot, it is placed in the left sub-array, if the element is larger than the pivot, it is placed in the right sub-array. (GeeksforGeeks, 2025b). This way, the sub-arrays will recursively be divided into smaller and smaller sub-arrays, until, finally the sub-array only contains one or zero elements. At which point, the dataset will be sorted.

In many examples found of the QuickSort algorithm, the pivot point is usually the last element in the list. This is because it is highly readable, it's clean and short and is usually implemented with a single for-loop. This approach is also called, “Lomuto partitioning scheme”, (GeeksforGeeks, 2024) and is also the implementation we decided to use in our algorithm when evaluating the four different pivot strategies and how they affect the algorithm's performance.

Implementation

```
29      @Override 4 usages
30      public void algorithm(double[] arr) {
31          nrOfSwaps = 0;
32          nrOfComparisons = 0;
33          if (arr == null || arr.length <= 1) return;
34          quickSortRecursive(arr, low: 0, high: arr.length - 1);
35      }
```

Line 29: Since the program utilizes an interface, and all algorithms implement it, we need to override the method.

Line 30: Method declaration, the method takes in the unsorted array as a parameter.

Line 31-32: These lines reset the swap and comparison counters. This is done here so that we make sure they do not get reset in the middle of a program, and so that the total doesn't add more than what is right.

Line 33: Checking if the array is empty or implicitly sorted if there's only one element left.

Line 34: Calling the **quickSortRecursive()** method, we also send the array, the first and last element in the array. These will also be used in the **partition()** method later.

```
37     public void quickSortRecursive(double[] arr, int low, int high){ 3
38         if (low < high) {
39             int pivotIndex = partition(arr, low, high, pivotStrategy);
40             quickSortRecursive(arr, low, high: pivotIndex - 1);
41             quickSortRecursive(arr, low: pivotIndex + 1, high);
42         }
43     }
```

Line 37: Method declaration, this method takes in the array, the first and last element of this array.

Line 38: This if-statement implicitly checks two things, whether the received array is empty or is one element long (which is already sorted). If this is the case, break out of the method.

Line 39: Here we call the **partition()** method with the array, first and last element in the array and the pivot strategy. The pivot strategy is sent as an argument from main. The result of this method call gives us the pivot index.

Line 40: This is a recursive method call, this line recursively delegates the sorting process by calling the very same method with the new sub-array, the first element and the value at the index right before the pivot.

Line 41: This is a recursive method call, this line recursively delegates the sorting process by calling the very same method with the new sub-array, the element right after the pivot and the last element.

```

45     private int partition(double[] arr, int low, int high, int pivotStrategy) { 1 usage
46         switch (pivotStrategy) {
47             case 1:
48                 //Case 1 declares the pivot as the first element in the list
49                 break;
50             case 2:
51                 //Swaps the first and last element in the list, so that the pivot is the last element
52                 swap(arr, low, high);
53                 break;
54             case 3:
55                 //Choosing a random pivot point
56                 int randomPivotIndex = low + random.nextInt( bound: (high - low) + 1);
57                 swap(arr, low, randomPivotIndex);
58                 break;
59             case 4:
60                 int mid = low + (high - low) / 2; //Finding the value at middle index
61                 int medianIndex = getMedianIndex(arr, low, mid, high); //Method finds the median value
62                 swap(arr, low, medianIndex); //Swaps the medianIndex with low so that pivot is the median
63                 break;
64             default:
65                 System.out.println("Invalid pivot strategy. Reverting to first element as pivot");
66                 break;
67         }

```

Line 45: Method declaration, the method takes in the array, the first and last element and the pivot strategy.

Line 46: The start of the switch-case, since we have four possible pivot strategies, we use a switch-case to allow all four.

Line 47-49: **Case 1**, if the pivot strategy is “1”, then do nothing; break out of the switch-case.

Line 50-53: **Case 2**, if the pivot strategy is “2”, it means that the last element is to be the pivot. We then call the **swap()** method to swap the first and last elements, because the pivot is always chosen as the first element. After calling the **swap()** method the last element is now the pivot.

Line 54-58: **Case 3**, if the pivot strategy is “3”, the pivot should be random. We do this by creating a Random object and calling that random object’s **nextInt()** method. We then pass in $(high - low) + 1$. This is because we need to stay within the bounds of the current array or sub-array.

Line 59-63: **Case 4**, in this case the pivot strategy is going to be a median-of-three. First we find the value at the middle index, we then call the helper-method **getMedianIndex()** and send the current array, the first, middle and last element in that array. The next step is again

to swap the medianIndex returned from the helper-method. Lastly we break out of the switch-case.

Line 64-66: **default**, this is reserved for if the incoming parameter (**pivotStrategy**) is not 1-4. If the program is run with anything else than the permitted value, the program defaults the strategy to "1". Side note: This logic is handled in the **LomutoQuickSort** class's constructor.

```
68         double pivot = arr[low];
69         int i = low;
70
71         for (int j = low + 1; j <= high; j++) {
72             nrOfComparisons++;
73             if (arr[j] <= pivot) {
74                 i++;
75                 swap(arr, i, j);
76             }
77         }
78         swap(arr, low, i);
79         return i;
80     }
```

Line 68: This line sets the pivot to the index of low. This is why the switch-case works for this code, no matter where the pivot ends up in the array, it is swapped with the index of low.

Line 69: Here we initialize a boundary "pointer" with the index of the pivot.

Line 71: A for-loop initializing j at the index after the pivot. As long as there's elements left in the array, j increments, meaning that each element is being checked.

Line 72: This line increments nrOfComparisons for our results.

Line 73-75: This if-statement checks if each element is smaller or equal to the value of pivot. If it is then we increment the boundary pointer i and then swap the value at i and j. This ensures that elements smaller than the pivot are moved to the left side of the pivot.

Line 78: This line calls the **swap()** method, we send the array, the first element and the boundary pointer as arguments. What this does is that it places the pivot in its final, sorted position.

Line 79: This is an important line, what we return here is the pivot index, which tells the recursive calls where to split the array.

```
82 @ private int getMedianIndex(double[] medianIndexList, int low, int mid, int high) {
83     double a = medianIndexList[low];
84     double b = medianIndexList[mid];
85     double c = medianIndexList[high];
86     nrOfComparisons++;
87     if(a > b){
88         nrOfComparisons++;
89         if(b > c) return mid;
90
91         nrOfComparisons++;
92         if(a > c) return high;
93         return low;
94     } else{
95         nrOfComparisons++;
96         if(a > c) return low;
97
98         nrOfComparisons++;
99         if(b > c) return high;
100         return mid;
101     }
102 }
```

Line 82: Method declaration, this is a helper method for finding the median index of the three possible

Line 83-85: Assigning the three indexes to variables for ease of writing the code.

Line 86, 88, 91, 95, 98: Incrementing the nrOfComparisons counter for each check that is performed. This ensures very accurate numbers for this report.

Line 87-101: In this if-else-statement we compare the first, middle and last index values with each other. By comparing them in each their own if-statement we can get accurate numbers on incrementing the nrOfComparisons variable.

```

104     private void swap(double[] arr, int index1, int index2) {
105         if (index1 != index2) {
106             double temp = arr[index1];
107             arr[index1] = arr[index2];
108             arr[index2] = temp;
109             nrOfSwaps++;
110         }
111     }

```

Line 104-111: Method declaration, this method takes in the array, and two indexes (index1 and index2). The **swap()** method first checks whether index1 and index2 are the same index or not. If they're not, index1 is placed in a temporary variable temp, the value of index2 is then placed into index1. The value placed in the temporary variable then replaces the value at index2. Lastly we increment the nrOfSwaps by one.

As stated in the introduction of this section, our Quicksort algorithm utilizes the Lomuto partition scheme. We chose this approach based on one observation regarding the dataset. Because we are asked to sort only unique values, we get to bypass Lomuto's main weakness, the time complexity nearing $O(n^2)$ when handling duplicates. With the duplicates removed, Lomuto actually outperformed Hoare's version in our empirical testing. Even though theoretically Hoare's version is supposed to perform better, we found that Lomuto finished sorting approximately 1000 nanoseconds faster than Hoare's, but with nearly a 100 more swaps. Now this presents an interesting discovery, normally swapping elements is a more expensive task to perform for the CPU rather than just comparing them. This may be a result of branch prediction, a modern CPU optimization strategy, which is way out of scope for this exam, but it provides a compelling explanation for why the outcomes were as they were.

Problem 4b: QuickSort Pivot Strategies

Pivot strategy	Elapsed time (nanoseconds)	Number of swaps	Number of comparisons
Pivot strategy 1 - First element	Original: 2930 Shuffled: 5423	Original: 293 Shuffled: 320	Original: 733 Shuffled: 740
Pivot strategy 2 - Last element	Original: 2874 Shuffled: 5317	Original: 383 Shuffled: 392	Original: 736 Shuffled: 744
Pivot strategy 3 - Random element	Original: 7511 Shuffled: 7854	Original: 362 Shuffled: 375	Original: 738 Shuffled: 742
Pivot strategy 4 - Median-of-three	Original: 3060 Shuffled: 5793	Original: 315 Shuffled: 336	Original: 816 Shuffled: 788

Does the number of comparisons change depending on the pivot strategy?

Yes. As we can see in the table above, the number of comparisons change depending on the strategy; pivot strategies 1, 2 and 3 are nearly identical when it comes to comparisons in both versions of the array. Shuffling the array clearly affects the algorithm's number of comparisons in pivot strategies 1, 2 and 3 in a negative way. Though not very much.

The big difference is shown when running the algorithm with median-of-three, with 816 comparisons. The increasing number of comparisons comes from the logic we implemented when finding the median, because right before every check in the **getMedianIndex()** method, we increment `nrOfComparisons`. We did this to get an accurate as possible number of comparisons. Interestingly, shuffling the array before sorting resulted in a fewer number of comparisons (816-788), this tells us that a randomized array helps the median approach find better pivots.

The elapsed time in all strategies tells the same story; shuffling the array before sorting takes longer than sorting the original array, though we can conclude that all pivot strategies maintain their $O(n \log n)$ time complexity.

Which pivot selection strategy works best for this dataset, and why?

For this dataset, we can see that pivot strategy 1, 2 and 4 perform quite well, whereas strategy 3 takes the longest to finish. At least when it comes to elapsed time. This is likely because utilizing the Random object to generate a random number for the pivot, requires the program to perform extra steps before it can partition the array, whereas simply selecting the first or last element is an instant action that requires no extra steps.

According to our testing, strategy 1 performed best, it has the lowest number of operations out of the four. Strategy 1 works best for this algorithm because the value chosen as the pivot happens to be close to the median value of the dataset. This creates close to evenly split sub-arrays, which in turn makes it easier to sort. If the pivot value happens to be either near the lowest value or near the highest value, the dividing of the array would be greatly unbalanced. This will cause the algorithm's best and average time complexity to degrade from its optimal $O(n \log n)$ to its $O(n^2)$ worst time complexity.

Theoretically, pivot strategy 4 (median-of-three) should have the best performance out of the pivot strategies, since it always chooses the median of the three values, the dividing of the array should be even and cause fewer operations. Though as seen in the video, the number of operations are roughly the same, but the elapsed time is different every time the algorithm is run. If the metric by which we conclude the best performing pivot strategy is the number of operations, then pivot strategy 1 is by far the better one. If the metric is elapsed time (nanoseconds), then sometimes pivot strategy 4 finishes first despite still having to do more operations.

References

Codecademy. (2025, April 17). *Time Complexity of Bubble Sort Explained with Examples*.
<https://www.codecademy.com/article/time-complexity-of-bubble-sort>

GeeksForGeeks. (2024, November 6). *Lomuto partition algorithm*.
<https://www.geeksforgeeks.org/dsa/lomuto-partition-algorithm/>

GeeksForGeeks. (2025a, October 3). *Merge Sort*.
<https://www.geeksforgeeks.org/dsa/merge-sort/>

GeeksForGeeks. (2025b, December 8). *Quick sort*.
<https://www.geeksforgeeks.org/dsa/quick-sort-algorithm/>

GeeksForGeeks. (2025c, July 23). *Time and Space Complexity of Insertion Sort*.
<https://www.geeksforgeeks.org/dsa/time-and-space-complexity-of-insertion-sort-algorithm/>

GeeksForGeeks. (2026a, February 26). *Insertion Sort Algorithm*.
<https://www.geeksforgeeks.org/dsa/insertion-sort-algorithm/>

GeeksForGeeks. (2026b, January 20). *Method Overloading in Java*.
<https://www.geeksforgeeks.org/java/method-overloading-in-java/>

Mytnik, A. (2024, September 1). *Warming up the JVM*. Medium.
https://medium.com/@anton_mytnik/warming-up-the-jvm-89101f8d9217

W3Schools. (n.d.-a). *DSA Bubble Sort*.
https://www.w3schools.com/dsa/dsa_algo_bubblesort.php

W3Schools. (n.d.-b). *DSA Insertion Sort*.
https://www.w3schools.com/dsa/dsa_algo_insertionsort.php

W3Schools. (n.d.-c). *DSA Insertion Sort Time Complexity*.
https://www.w3schools.com/dsa/dsa_timecomplexity_insertionsort.php

W3Schools. (n.d.-d). *DSA Merge Sort*.
https://www.w3schools.com/dsa/dsa_algo_mergesort.php

W3Schools. (n.d.-e). *DSA Merge Sort Time Complexity*.
https://www.w3schools.com/dsa/dsa_timecomplexity_mergesort.php